

BURP SUITE

ACADEMY / RAPPORT DE LABORATOIRE

Tests d'API et exploitation de la documentation

Découverte d'un endpoint DELETE et suppression de carlos

Étudiant	Gana Faye
Formation	Sécurité des applications web
Plateforme	PortSwigger Web Security Academy
Laboratoire	Exploiting an API endpoint using documentation
Module	API Testing
Niveau	Apprentice
Identifiants	wiener:peter
Date	24 juillet 2026
Niveau de risque	Élevé

Document réalisé dans un environnement de laboratoire autorisé à des fins pédagogiques.

Table des matières

1	Introduction et objectifs	2
1.1	Contexte	2
1.2	Objectifs	2
1.3	Authentification	2
2	Reconnaissance du point d'entrée API	3
2.1	Modification légitime de l'adresse électronique	3
2.2	Transmission vers Burp Repeater	4
3	Découverte de la documentation de l'API	5
3.1	Suppression de l'identifiant utilisateur	5
3.2	Suppression du segment de ressource	6
3.3	Accès à la racine avec la méthode GET	8
3.4	Affichage dans le navigateur	9
4	Exploitation et validation	10
4.1	Sélection de l'opération DELETE	10
4.2	Validation officielle du laboratoire	11
5	Analyse de la vulnérabilité	12
5.1	Chaîne d'exploitation	12
5.2	Classification	12
5.3	Impact	13
6	Recommandations de remédiation	13
6.1	Contrôle d'accès sur chaque opération	13
6.2	Protection de la documentation	13
6.3	Durcissement des routes	13
7	Conclusion	14

1 Introduction et objectifs

1.1 Contexte

Les applications web modernes utilisent fréquemment une API pour séparer l'interface utilisateur de la logique métier. L'interface graphique n'expose cependant pas toujours toutes les opérations disponibles. Une documentation publique, une ancienne route ou une méthode HTTP inutilisée par le client peut révéler une fonction sensible directement callable.

Le laboratoire *Exploiting an API endpoint using documentation* porte sur la reconnaissance d'une API REST. L'application permet à un utilisateur authentifié de modifier son adresse électronique. Cette action fournit un premier endpoint à partir duquel la structure de l'API peut être explorée.

1.2 Objectifs

La démarche vise à :

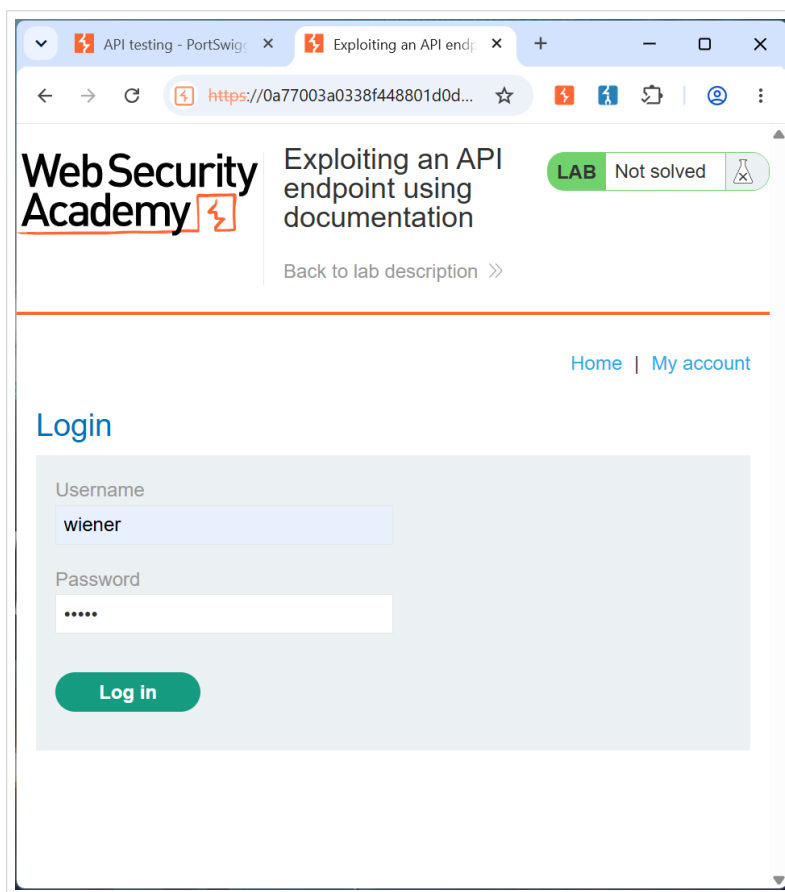
- s'authentifier avec le compte de laboratoire `wiener:peter` ;
- intercepter l'appel API utilisé pour modifier l'adresse électronique ;
- analyser progressivement les segments de l'URI ;
- découvrir la racine de documentation `/api/` ;
- identifier la méthode `DELETE` documentée ;
- supprimer l'utilisateur `carlos` et valider le laboratoire.

Périmètre autorisé

Toutes les opérations présentées ont été réalisées dans un laboratoire PortSwigger isolé. Une méthode destructive telle que `DELETE` ne doit être testée que sur une ressource explicitement prévue par le scénario ou couverte par une autorisation.

1.3 Authentification

La première étape consiste à ouvrir la page de connexion et à utiliser les identifiants fournis. L'interface indique que le laboratoire n'est pas encore résolu.

FIGURE 1 – Connexion au laboratoire avec le compte **wiener**.

2 Reconnaissance du point d'entrée API

2.1 Modification légitime de l'adresse électronique

Une fois connecté, l'adresse électronique du profil est modifiée depuis l'interface. L'historique HTTP de Burp Proxy révèle une requête **PATCH** adressée à l'endpoint suivant :

```
PATCH /api/user/wiener HTTP/2
Host: <laboratoire>.web-security-academy.net
Cookie: session=<valeur>
Content-Type: text/plain; charset=UTF-8

{
  "email": "wiener@normal-user.net"
}
```

La réponse 200 OK retourne un objet JSON contenant le nom d'utilisateur et la nouvelle adresse. Cette observation confirme plusieurs éléments : le préfixe **/api**, la ressource **user**, l'identifiant **wiener** et la prise en charge de la méthode **PATCH**.

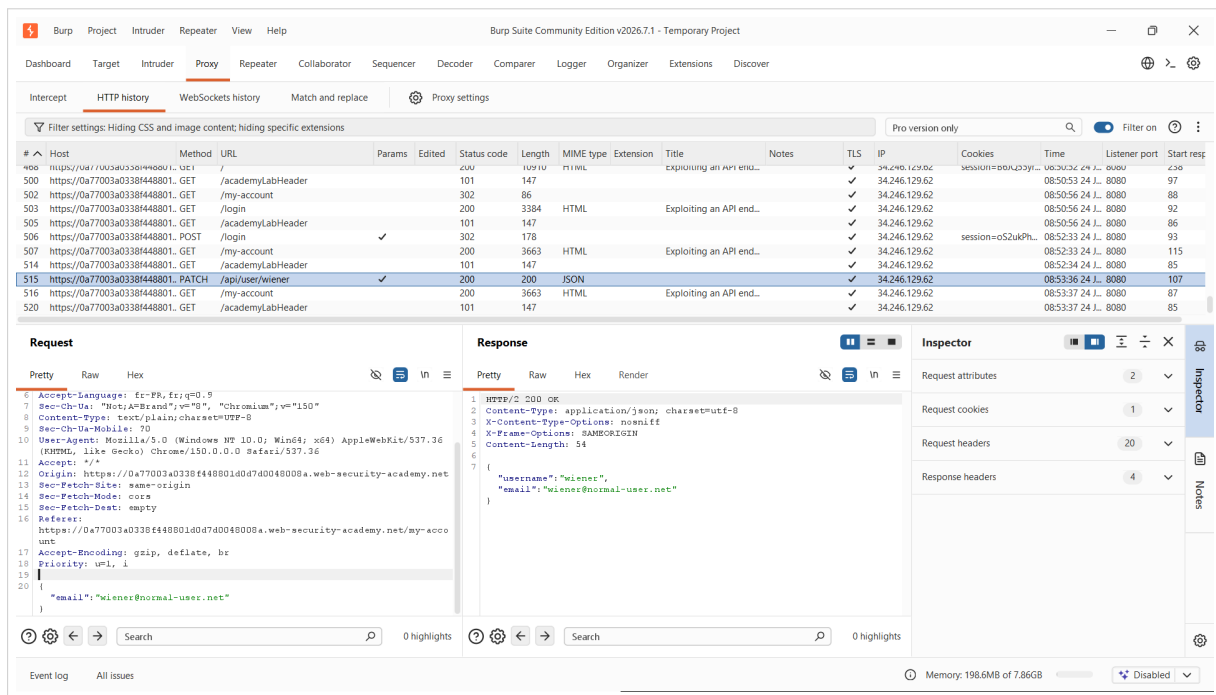


FIGURE 2 – Historique Burp Proxy : interception de PATCH /api/user/wiener et réponse JSON 200 OK.

2.2 Transmission vers Burp Repeater

La requête est envoyée vers Burp Repeater afin de modifier séparément les segments du chemin. Repeater conserve la session, les en-têtes et le corps, ce qui permet d'attribuer les différences de réponse aux changements d'URI.

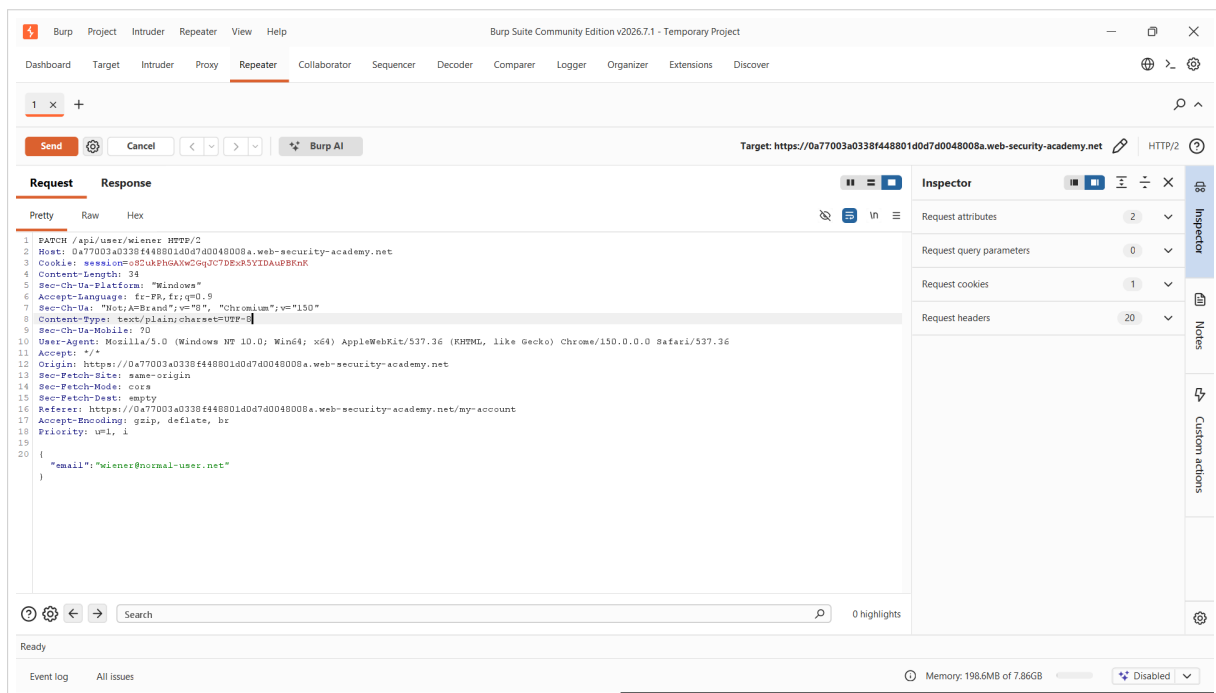


FIGURE 3 – Requête légitime transférée dans Burp Repeater avant les tests de chemin.

/api/user/wiener → /api/user → /api → /api/

3 Découverte de la documentation de l'API

3.1 Suppression de l'identifiant utilisateur

Le premier test retire uniquement le dernier segment **wiener**. La requête devient :

```
PATCH /api/user HTTP/2
```

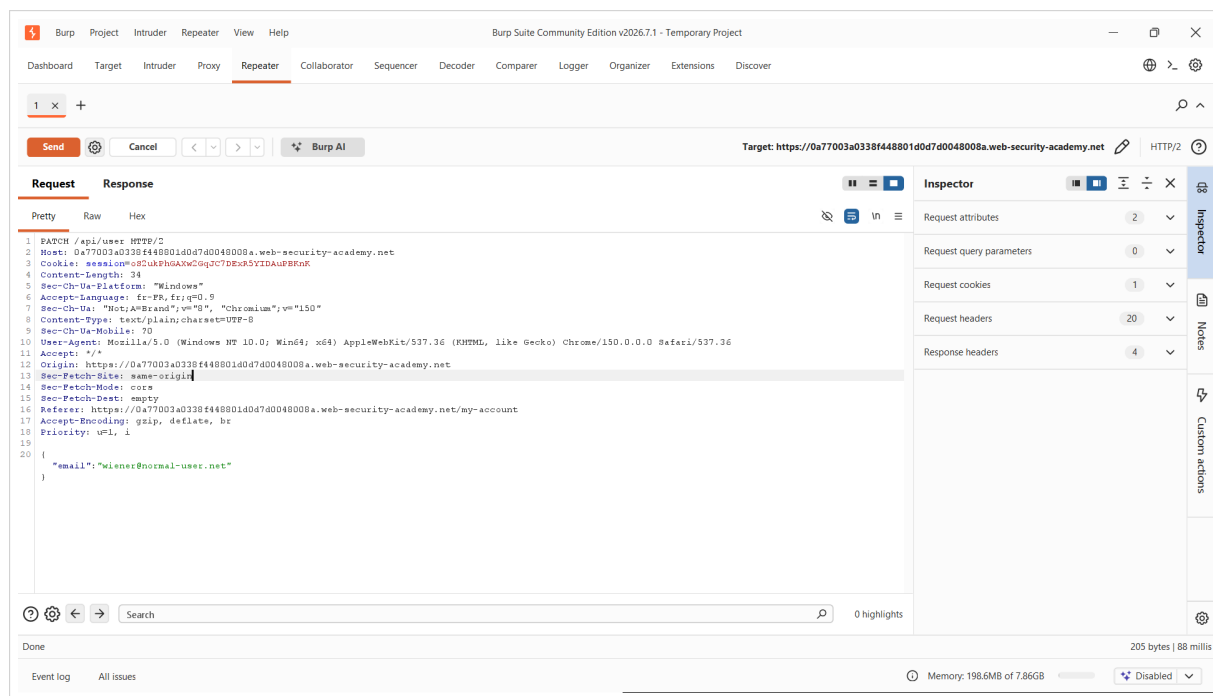


FIGURE 4 – Première variation : suppression de l'identifiant **wiener** dans le chemin.

Le serveur répond par 400 Bad Request et fournit un message particulièrement instructif :

```
HTTP/2 400 Bad Request
Content-Type: application/json; charset=utf-8

{
  "error": "Malformed URL: expecting an identifier"
}
```

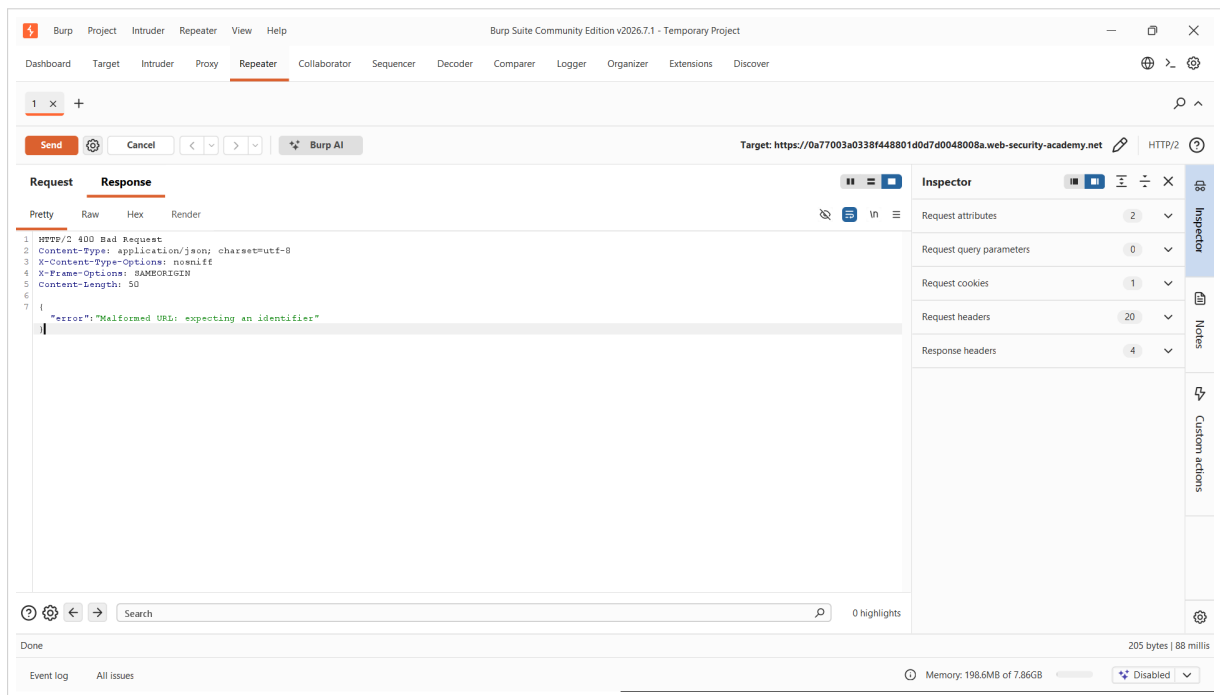


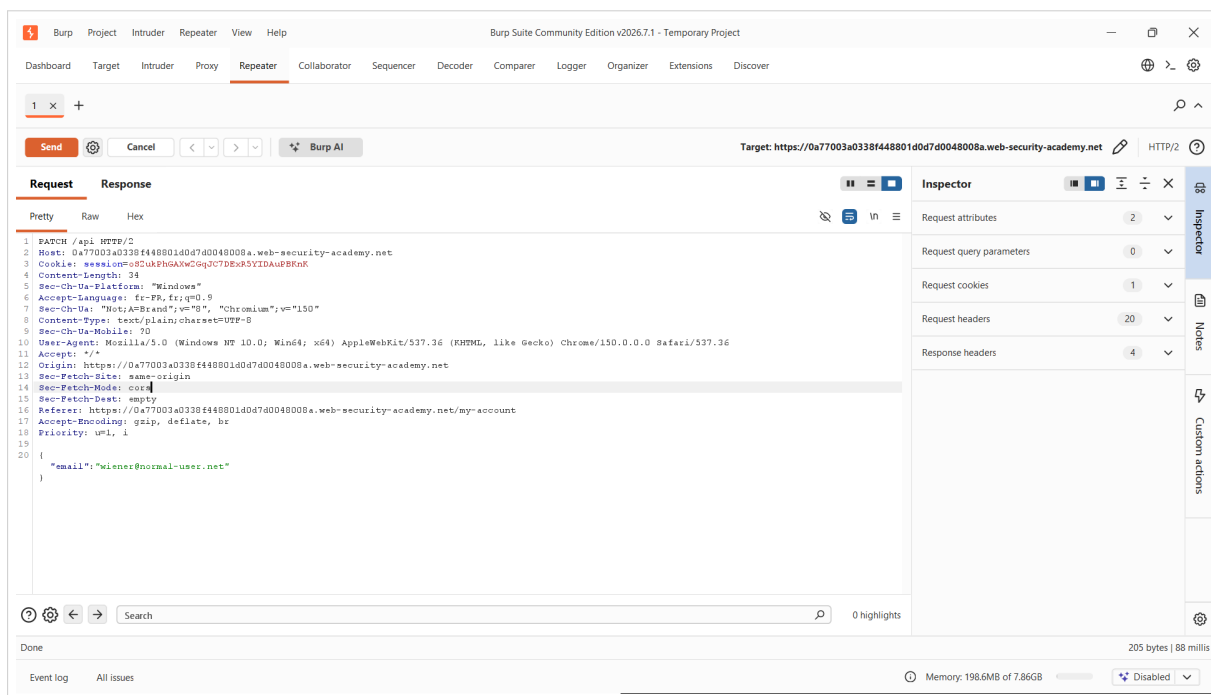
FIGURE 5 – Réponse d'erreur indiquant que la route `/api/user` attend un identifiant.

Le message confirme que `user` représente une ressource paramétrée. Il ne révèle pas encore la documentation, mais valide la structure `/api/user/{identifiant}`.

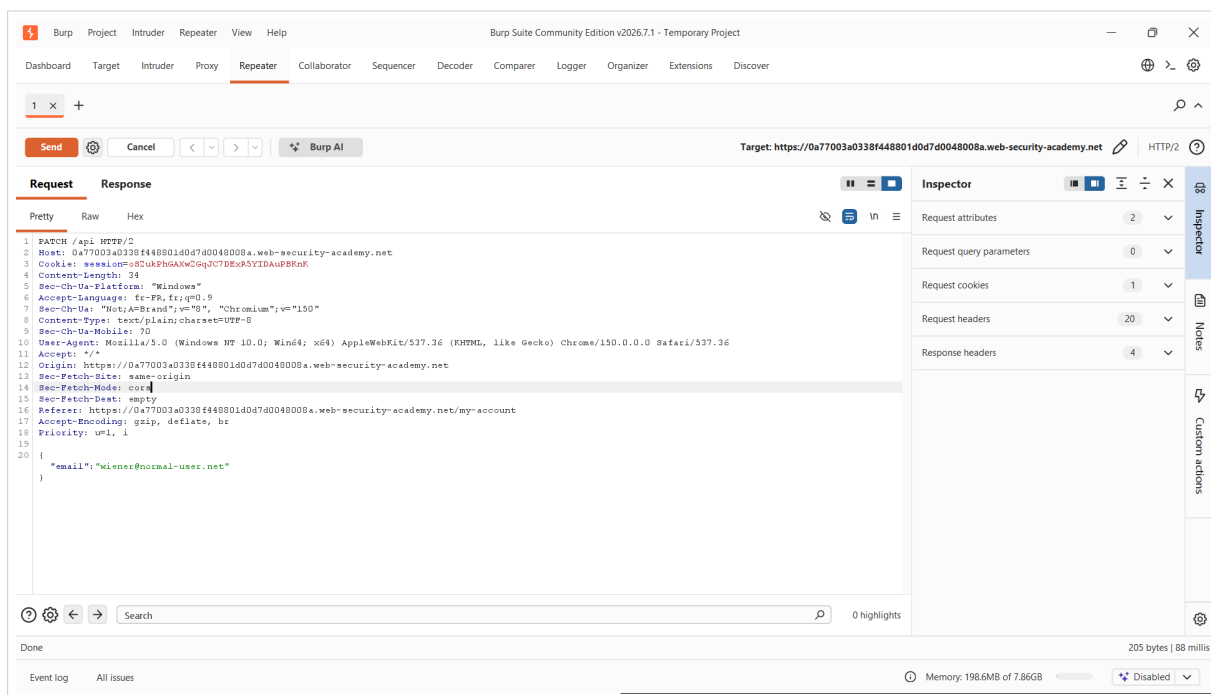
3.2 Suppression du segment de ressource

Le segment `user` est ensuite retiré afin de tester la racine `/api`. La requête conserve provisoirement la méthode `PATCH` et le corps `JSON` :

```
PATCH /api HTTP/2
```

FIGURE 6 – Deuxième variation du chemin : test de la racine `/api`.

La capture suivante documente la même tentative dans Repeater avant l'analyse de la réponse.

FIGURE 7 – Requête `PATCH /api` après suppression du segment `user`.

Le serveur renvoie une redirection :

```

HTTP/2 302 Found
Location: /api/
Content-Length: 0

```

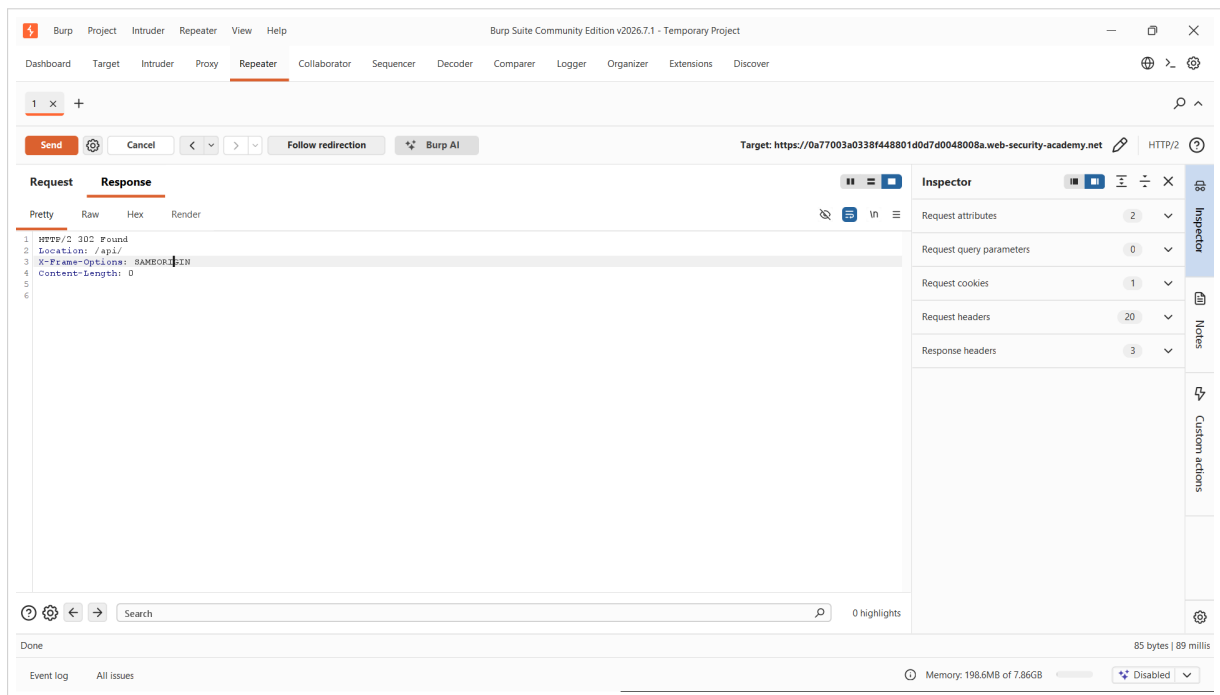


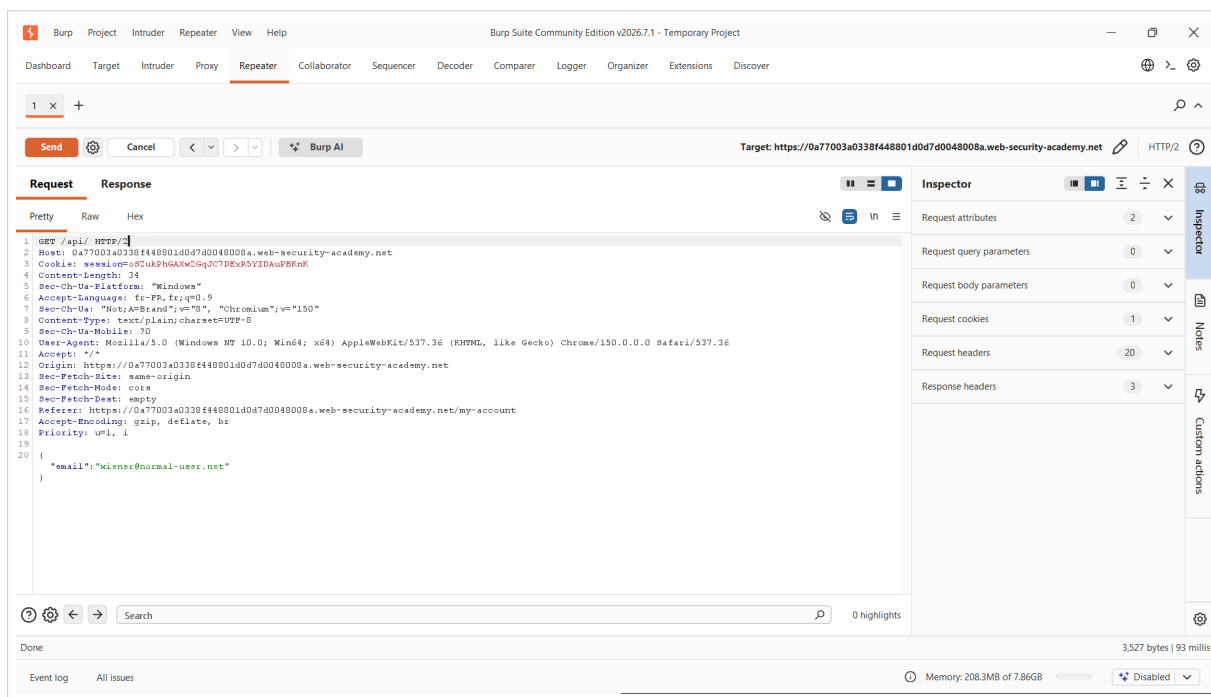
FIGURE 8 – Réponse 302 Found redirigeant explicitement vers la racine /api/.

Cette réponse constitue l'indice décisif : une ressource existe avec une barre finale. La redirection normalise le chemin et révèle l'emplacement de la documentation.

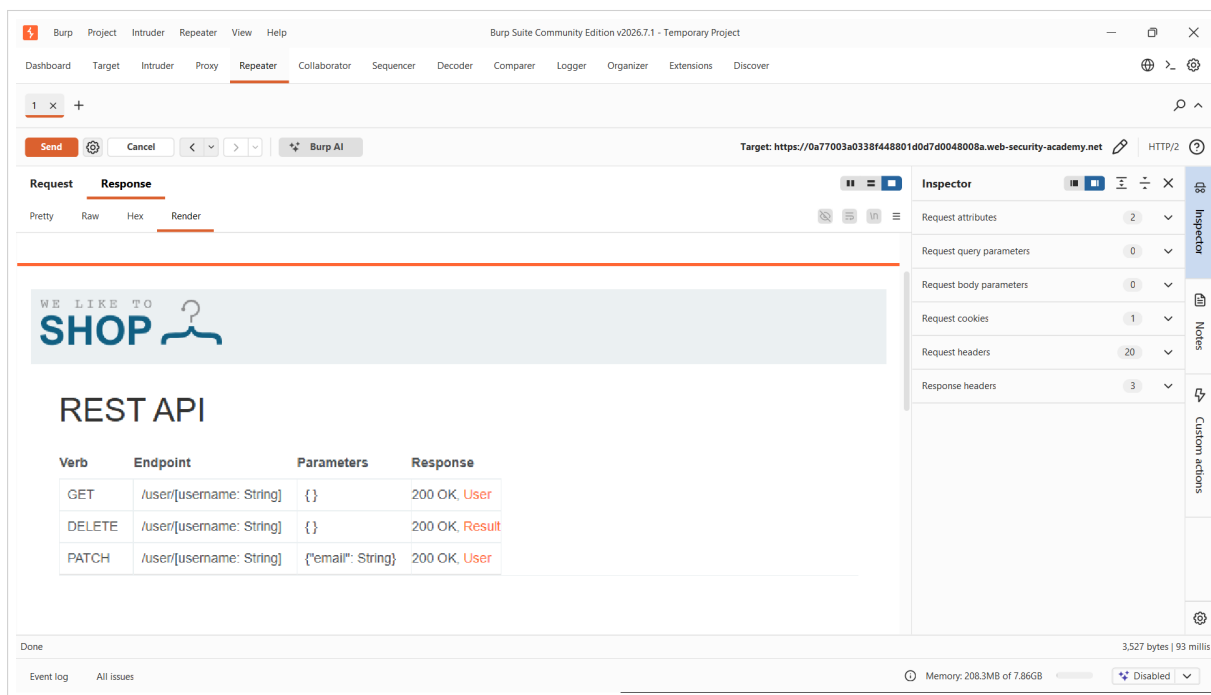
3.3 Accès à la racine avec la méthode GET

La méthode est remplacée par GET, le corps n'étant plus nécessaire :

```
GET /api/ HTTP/2
Host: <laboratoire>.web-security-academy.net
Cookie: session=<valeur>
```

FIGURE 9 – Requête GET `/api/` construite dans Burp Repeater.

La réponse 200 OK, rendue dans Burp, contient une page HTML intitulée *REST API*. Elle présente un tableau de routes et de méthodes.

FIGURE 10 – Rendu de la documentation REST retournée par GET `/api/`.

3.4 Affichage dans le navigateur

La fonction *Show response in browser* permet d'ouvrir la réponse complète dans le navigateur. La documentation interactive expose trois opérations sur la ressource utilisateur :

Méthode	Endpoint	Fonction
GET	/user/[username: String]	Consulter un utilisateur
DELETE	/user/[username: String]	Supprimer un utilisateur
PATCH	/user/[username: String]	Modifier l'adresse électronique

TABLE 1 – Opérations révélées par la documentation

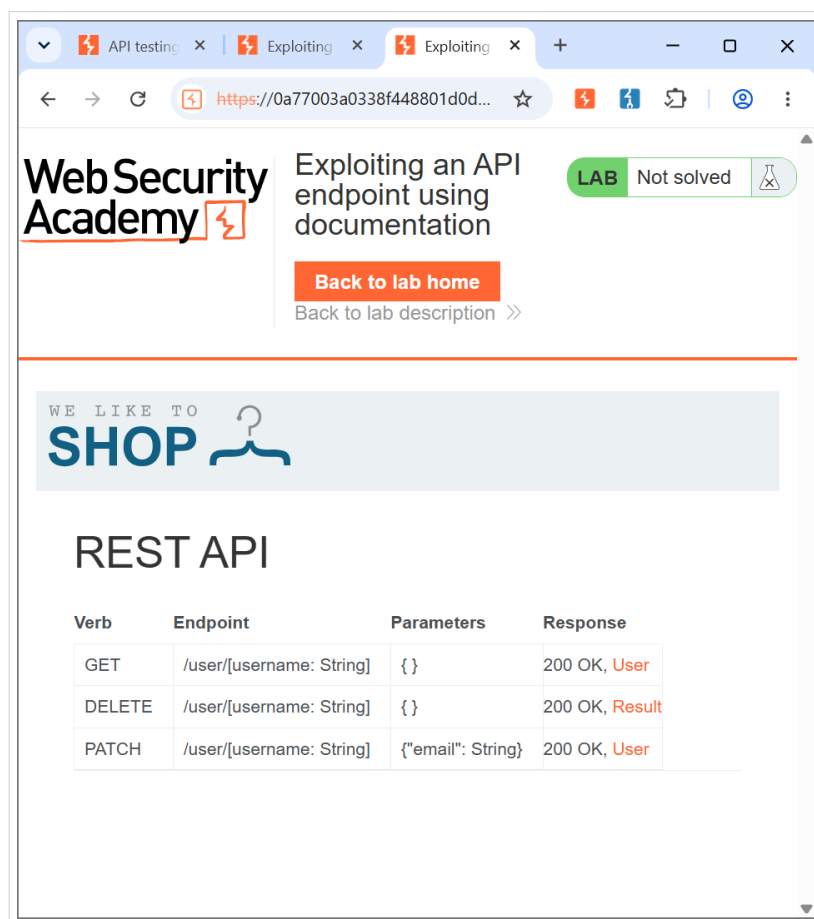


FIGURE 11 – Documentation interactive de l'API affichée dans le navigateur.

Constat de sécurité

L'interface principale n'offrait pas de fonction de suppression, mais la documentation publique révèle un endpoint DELETE. L'exposition de la documentation facilite la découverte ; l'impact réel provient surtout de l'absence de contrôle d'autorisation empêchant un utilisateur standard de supprimer un autre compte.

4 Exploitation et validation

4.1 Sélection de l'opération DELETE

L'opération DELETE est ouverte depuis la documentation interactive. Le champ `username` reçoit la valeur `carlos`. L'interface génère la requête correspondante :

```
DELETE /api/user/carlos HTTP/2
Host: <laboratoire>.web-security-academy.net
Cookie: session=<valeur>
```

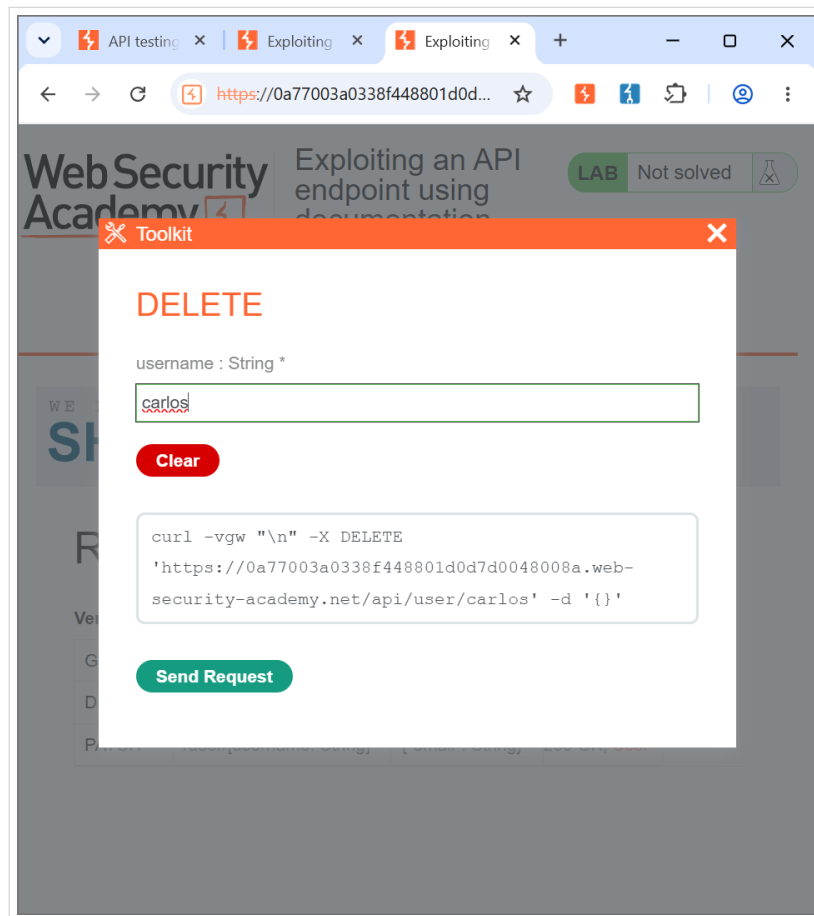


FIGURE 12 – Préparation de la suppression de **carlos** depuis l'outil interactif de la documentation.

La méthode destructive est envoyée avec la session de l'utilisateur **wiener**. L'API ne vérifie pas que ce compte possède un privilège administratif suffisant pour supprimer un autre utilisateur.

4.2 Validation officielle du laboratoire

Après l'exécution, l'interface affiche l'état *Solved* ainsi que le message *Congratulations, you solved the lab !*. Cette preuve confirme que le compte **carlos** a été supprimé et que l'objectif du laboratoire est atteint.

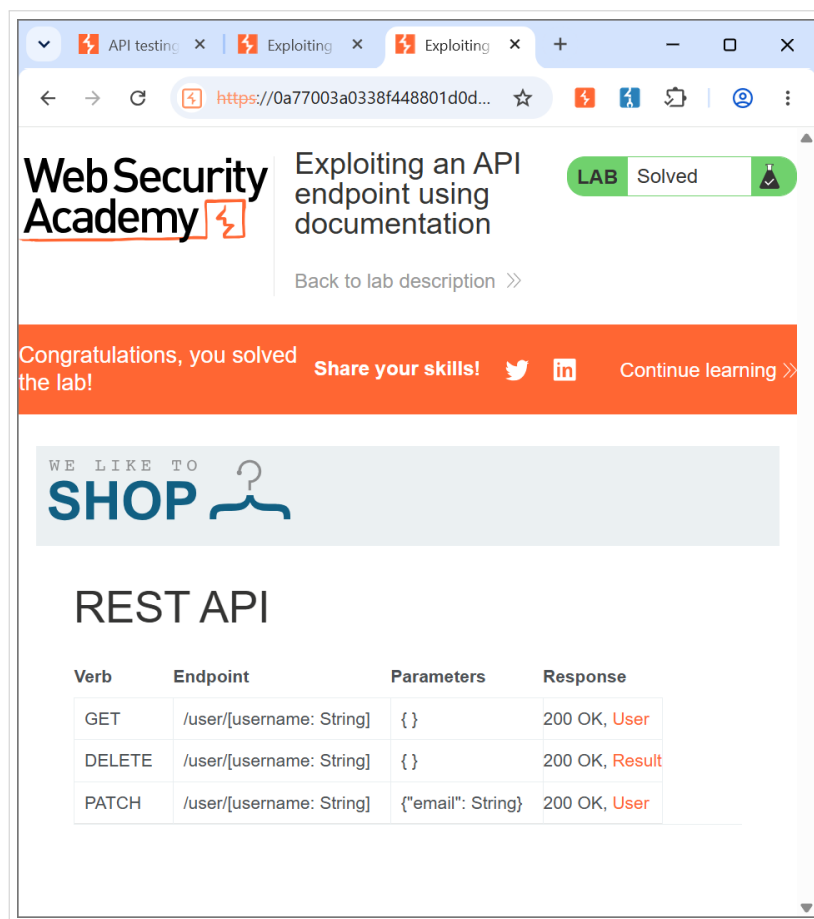


FIGURE 13 – Validation finale : laboratoire résolu après la suppression de l'utilisateur carlos.

5 Analyse de la vulnérabilité

5.1 Chaîne d'exploitation

Étape	Résultat
Observation	Découverte de PATCH /api/user/wiener.
Réduction du chemin	Identification du modèle /api/user/{identifiant}.
Test de la racine	Redirection 302 vers /api/.
Documentation	Révélation des méthodes GET, PATCH et DELETE.
Exploitation	Appel de DELETE /api/user/carlos.
Impact	Suppression non autorisée d'un autre utilisateur.

TABLE 2 – Synthèse de la chaîne d'exploitation

5.2 Classification

La faiblesse principale correspond à une autorisation fonctionnelle insuffisante. Un utilisateur authentifié à faibles privilèges peut invoquer une fonction destructive qui devrait être réservée à un

administrateur. La valeur `carlos` désigne également un objet appartenant à un autre utilisateur, ce qui ajoute une dimension d'autorisation au niveau de l'objet.

La documentation publique augmente la facilité d'exploitation, mais sa suppression ne corrigerait pas le défaut de contrôle d'accès sous-jacent. Un attaquant pourrait toujours découvrir la méthode par analyse du code client, fuzzing ou essais de méthodes HTTP.

5.3 Impact

- **Intégrité** : suppression d'un compte sans privilège administratif ;
- **Disponibilité** : impossibilité pour l'utilisateur supprimé d'accéder à son compte ;
- **Confidentialité** : la documentation révèle la structure de l'API et ses capacités ;
- **Abus à grande échelle** : la méthode pourrait être automatisée contre plusieurs utilisateurs si aucune limite supplémentaire n'existe.

6 Recommandations de remédiation

6.1 Contrôle d'accès sur chaque opération

L'API doit appliquer une politique *deny by default*. La méthode DELETE doit exiger un rôle ou une permission explicite, indépendamment de l'interface qui l'appelle.

- vérifier l'identité et le rôle pour chaque méthode HTTP ;
- contrôler la relation entre l'utilisateur et l'objet ciblé ;
- séparer les routes d'administration des opérations utilisateur ;
- tester une matrice comptes, rôles, méthodes et objets ;
- journaliser toute suppression avec l'identité de l'auteur.

6.2 Protection de la documentation

La documentation interactive de production doit être désactivée lorsqu'elle n'est pas nécessaire ou protégée par une authentification administrative forte. Les fichiers OpenAPI ne doivent pas contenir de secrets, d'URLs internes ou d'exemples sensibles.

Cette mesure réduit la surface de reconnaissance, mais elle ne remplace jamais les contrôles d'autorisation sur les endpoints.

6.3 Durcissement des routes

- désactiver les méthodes et versions inutilisées ;
- maintenir un inventaire des routes réellement déployées ;
- retourner des erreurs cohérentes sans détails internes excessifs ;
- valider strictement les identifiants et le corps des requêtes ;

- utiliser une protection contre l'automatisation et des limites de débit adaptées ;
- mettre en place des tests négatifs d'autorisation dans le pipeline.

7 Conclusion

Le laboratoire a démontré qu'une action légitime de modification de profil pouvait servir de point de départ à la cartographie d'une API. La réduction progressive du chemin `/api/user/wiener` a révélé une redirection vers `/api/`, puis une documentation REST exposant une méthode DELETE absente de l'interface principale.

L'appel de DELETE `/api/user/carlos` avec la session du compte standard `wiener` a supprimé l'utilisateur et validé le laboratoire. La correction prioritaire consiste à imposer une autorisation fonctionnelle et objet sur chaque opération. La protection de la documentation et la suppression des routes inutiles constituent des mesures complémentaires de réduction de surface.